CS612 Lecture 07 — In-Class Exercises

Dynamic Programming II — Optimization & Pseudo-Polynomial Algorithms · Sat 07 Nov 2026 · ungraded

A — Model it

1. Weighted interval scheduling: show with explicit instances that greedy by **largest value** and greedy by **largest value per unit length** both fail.
2. **Russian dolls.** Envelopes (w_i, h_i); one fits inside another iff both dimensions are strictly smaller. Find the longest nesting chain in $O(n \log n)$.
3. Modify the $O(n^2)$ LIS DP to count the **number** of longest increasing subsequences. What must each state store?

B — Which knapsack algorithm?

Instance	n	W	values
(a)	200	10^5	up to 10^9
(b)	200	10^{12}	integers ≤ 100
(c)	40	10^{15}	up to 10^{15}
(d)	5000	10^{12}	up to 10^{12}; weights ≈ values ± 1%

For each: the method and its cost, justified by the dominant term.

C — Midterm-style practice

1. **Optimal BST.** Keys $k_1 < \cdots < k_n$ with search frequencies f_i; a search for k_i costs its depth + 1. Build the minimum-total-cost BST: state, recurrence, base cases, order, complexity, and why the "choice of root" case is correct.
2. **Coin change (minimum coins)** for coin values $c_1..c_k$ and target T: give the DP. Is it polynomial? Give a coin system for which greedy is not optimal.
3. True or false: "Subset sum with n numbers, each at most n^2, is solvable in polynomial time." Justify precisely.
4. Edit distance for two strings of length 10^6: what can you compute in 16 MB of memory, and how?

Take-home (optional)

Run `Student_Resources/dp2_bench.cpp`. (1) Implement the Pareto-list knapsack and compare its running time with the $O(nW)$ DP on the same instances. (2) Generate strongly correlated instances ($v_i = w_i + 10$) and repeat. What happens to the Pareto-list size?